

Step 3 — Deciding What's Worth Remembering, and Writing It Back — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (a real structured-output judgment call), **Advanced** (the dedupe design, and testing "says no" as carefully as "says yes"). Read Basic first even if structured output is already familiar — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Judging one message, not the whole conversation](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Judging one message, not the whole conversation

Basic Version

After the model answers, look at what the *user* just said (not the bot's reply) and ask a separate, small question: is there a fact in there worth remembering forever? Most of the time the answer is no. Only write to the Store when the answer is genuinely yes.

Things to use: - A pydantic model with `worth_remembering: bool`, `key: str | None`, `value: str | None`. - `model.with_structured_output(YourModel)` — this is how you get a guaranteed, typed answer back instead of parsing free text yourself. - A short, fixed list of allowed `key` values, written directly into the judge's prompt — not left for the model to invent a new one each time.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exact pieces:

- `from pydantic import BaseModel` — define `class MemoryJudgment(BaseModel):`
`worth_remembering: bool; key: str | None = None; value: str | None = None` .
- `judge_model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(MemoryJudgment)` — a **separate** `ChatOpenAI` instance from the one `chat_node` uses, since this is a different job with a different prompt, even though it's the same underlying model.
- A prompt that names the allowed keys explicitly: `"preferred_name"` for what to call the user, `"stated_preference"` for a general like/dislike or standing instruction — and gives 2-3 worked examples of each, plus 2-3 examples of things that are **not** worth saving (today's mood, a one-off request, small talk).
- `judgment = judge_model.invoke([{"role": "user", "content": prompt_text}])` returns a `MemoryJudgment` instance directly — no manual JSON parsing needed.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The real design question this step exists to make you answer: how do you stop "I'm Sam" and "call me Sam" and "actually it's Sam, not Samuel" from becoming three separate, half-contradicting rows in the Store? The answer isn't a smarter judge — it's a **small, fixed set of keys**, decided up front, that every save has to fit into. `"preferred_name"` is always the key for a name preference, no matter how the user phrases it. `store.put(namespace, key, value)` **overwrites** whatever was at that key before — that overwrite is the entire dedupe mechanism. You don't need fuzzy matching, embeddings, or a "is this similar to an existing fact?" check for a project this size; you need the judge to map varied phrasing onto the same small set of keys, consistently.

This means the judge's job is really two decisions bundled into one call: "is this worth remembering at all," and if so, "which of my known keys does this belong to." Write the prompt so both are explicit — don't just ask "extract a fact," ask "extract a fact **and** classify it under one of these exact key names: ...".

Also design the "no" case as carefully as the "yes" case. A judge that's a little too eager to say yes will slowly fill the Store with noise ("user said they're tired today" saved as if it were durable) — which defeats the entire point of Step 2's `recall_node`, since a system prompt stuffed with yesterday's mood is worse than one with nothing at all. Test "no" on purpose, the same way you tested "yes."

Things to try before Hint 2: - Feed the judge "I think I'll have pasta tonight" and confirm `worth_remembering` comes back `False`. - Feed it "I'm Sam" then, in a later call, "actually, call me Sammy" — confirm both map to the same key (`preferred_name`), so the second `store.put` overwrites the first instead of creating a second entry.

Difference between Basic, Intermediate, and Advanced: Basic names the idea (judge one message, save rarely). Intermediate gives the real structured-output setup and prompt shape. Advanced is the actual answer to the dedupe question this step is built around: a small fixed key set plus `put`'s natural overwrite behavior, not a fancier judge or a similarity search.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
judge_message(text):
    ask a small model: "does this contain a durable fact? which known key
    does it belong to (preferred_name, stated_preference, or none)? what's
    the value?"
    return the answer as a typed object

write_back_node(state, store):
    look at the user's latest message
    judgment = judge_message(that text)
    if judgment.worth_remembering:
        store.put(namespace_for(state["user_id"]), judgment.key, {"text": judgment.text})
    return state unchanged (this node doesn't touch messages)

graph: START -> recall_node -> chat_node -> write_back_node -> END
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Here's almost the whole `memory_writer.py` — type it out yourself and adjust the examples to your own scenario:

```
# memory_writer.py
from langchain_openai import ChatOpenAI
from pydantic import BaseModel

ALLOWED_KEYS = ["preferred_name", "stated_preference"]

JUDGE_PROMPT = """Decide if this message contains a fact worth remembering forever
about the user, versus something that only matters today.

Worth remembering (examples): "I'm Sam", "call me Sam, not Samuel",
"I prefer short answers", "always write my emails formally".
NOT worth remembering (examples): "I think I'll have pasta tonight",
"I'm tired today", "thanks", "ok sounds good".

If it IS worth remembering, classify it under exactly one of these keys:
{allowed_keys}

Message: {text}
"""

class MemoryJudgment(BaseModel):
    worth_remembering: bool
    key: str | None = None
    value: str | None = None

judge_model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(MemoryJudgment)

def judge_message(text: str) -> MemoryJudgment:
    prompt = JUDGE_PROMPT.format(allowed_keys=", ".join(ALLOWED_KEYS), text=text)
    return judge_model.invoke([{"role": "user", "content": prompt}])
```

Your turn: write `write_back_node(state, store)` in `graph.py`, wiring it in after `chat_node`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the actual write logic yourself, using this skeleton:

```
# graph.py (relevant part only)
import logging
from functools import partial
from memory_writer import judge_message
from store import namespace_for

logger = logging.getLogger(__name__)

def write_back_node(state, store) -> dict:
    last_user_message = None
    for message in reversed(state["messages"]):
        if getattr(message, "type", None) == "human" or (isinstance(message, dict)
            last_user_message = message
            break
    if last_user_message is None:
        return {}

    text = last_user_message.content if hasattr(last_user_message, "content") else
    judgment = judge_message(text)

    if not judgment.worth_remembering or not judgment.key:
        logger.debug("write_back_node: nothing worth saving in %r", text)
        return {}

    # your turn: call store.put with the right namespace, key, and value shape,
    # and log an INFO line naming which key was written (not the raw user text,
    # to keep logs from becoming a second, uncontrolled copy of user data)
    ...

    return {}
```

Think about that last comment before you fill it in: logging `key` (like `"preferred_name"`) is useful for debugging. Logging the raw `value` at `INFO` level means your log files become a second, unmanaged copy of exactly the kind of personal data Step 4's `forget_me` is supposed to erase — worth avoiding on purpose, not by accident.

Compare your finished `memory_writer.py` and `write_back_node` against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic sketches the judge-then-maybe-save flow in plain words. Intermediate gives the real structured-output judge, with concrete worth-saving and not-worth-saving examples baked into the prompt. Advanced gives the actual node wiring, including finding the right message to judge and the privacy point about what a write-back node should and shouldn't log.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)